

SOLID Principles for Object-Oriented Design

- Martin and Martin, Agile Principles, Patterns, and Practices in C#, Prentice Hall, 2006. (The PPP book)
- <http://butunclebob.com/ArticleS.UncleBob.PrinciplesOfOod>
- <https://people.engr.tamu.edu/choe/choe/courses/20fall/315/lectures/slide23-solid.pdf>

SOLID

- SRP The Single Responsibility Principle

A class should have one, and only one, reason to change.

- OCP The Open Closed Principle

You should be able to extend a classes behavior, without modifying it.

- LSP The Liskov Substitution Principle

Derived classes must be substitutable for their base classes.

SOLID

- ISP The Interface Segregation Principle

Make fine grained interfaces that are client specific.

- DIP The Dependency Inversion Principle

Depend on abstractions, not on concretions.

History of SOLID

Robert C. Martin is the main person behind these ideas (some individual ideas predate him though).

- First appeared as a news group posting in 1995.
- Full treatment given in Martin and Martin, Agile Principles, Patterns, and Practices in C#, Prentice Hall, 2006. (The PPP book)

Benefits of SOLID

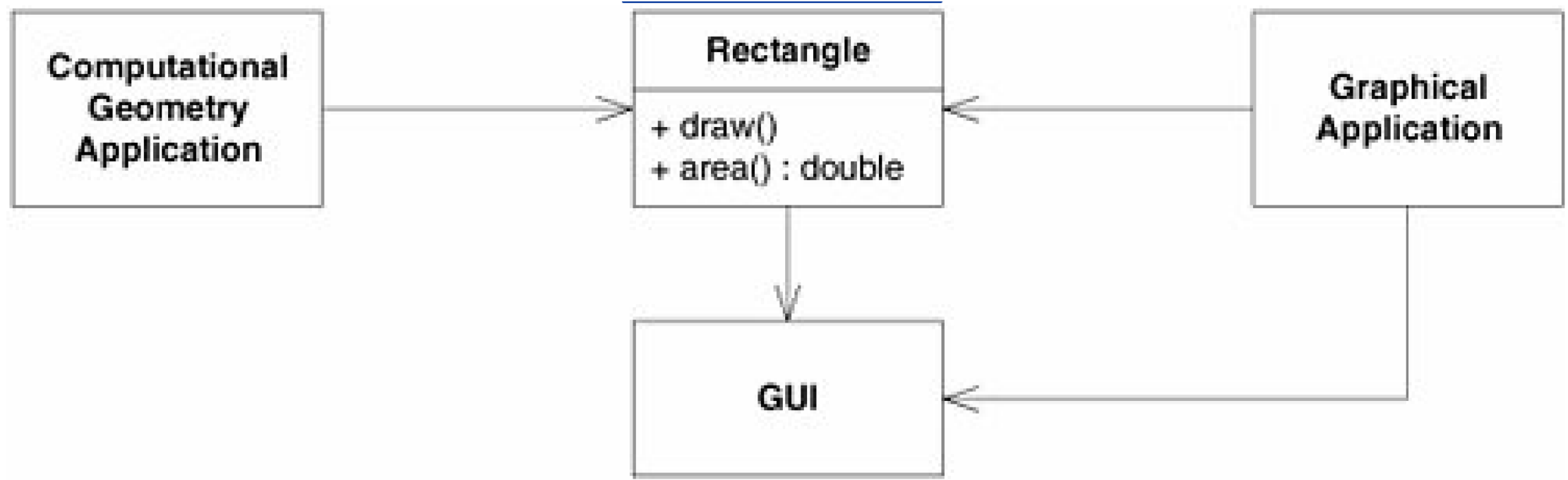
- Provides a principled way to manage dependency.
- Serves as a solid foundation for OOD upon which more complicated design patterns can be built upon and incorporated naturally.
- Results in code that are flexible, robust, and reusable.

SRP: The Single Responsibility Principle

A class should have one, and only one, reason to change.

SRP : Rectangle

Example: Rectangle class with draw() and area()



SRP: Rectangle

This design violates SRP. The Rectangle class has two responsibilities:

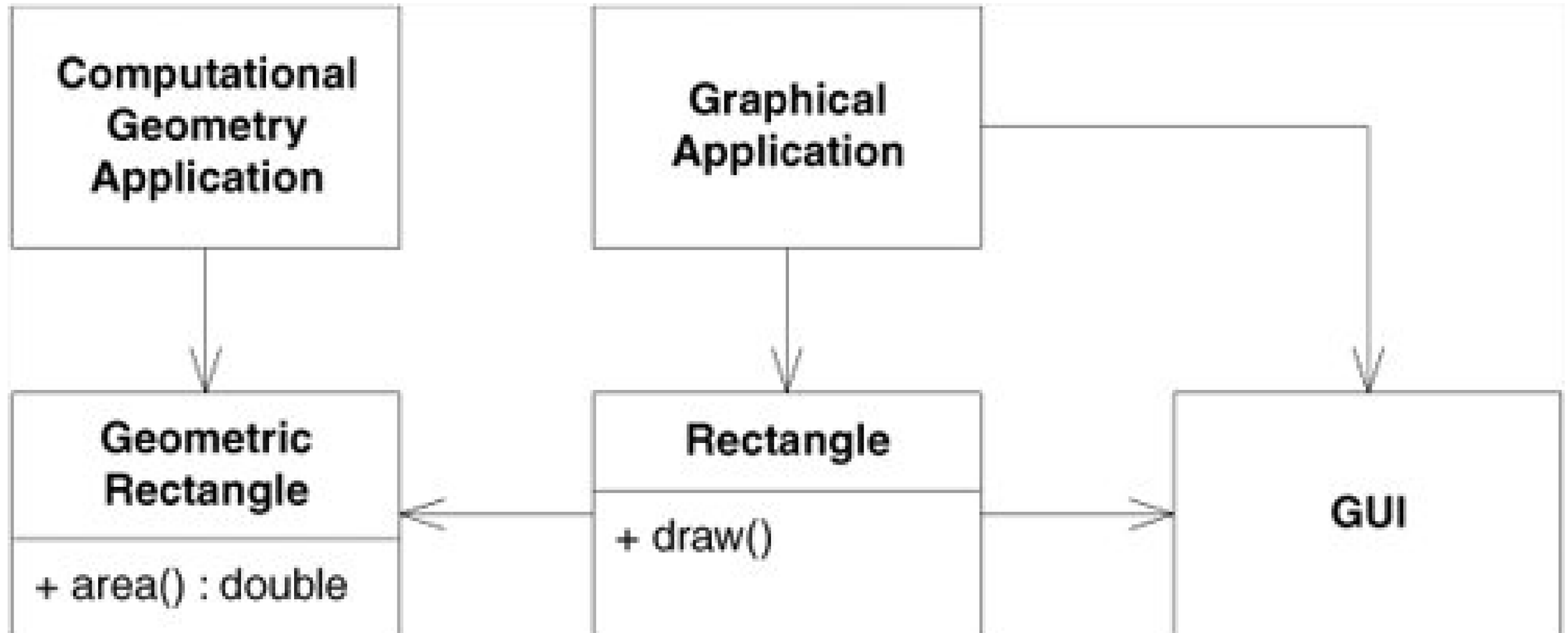
- Provide a mathematical model of the geometry of a rectangle.
- Render the rectangle on a GUI.

And will result in the following issues:

- Computational geometry now depends on GUI, via Rectangle.
- Any changes to Rectangle due to Graphical application necessitates rebuild, retest, etc. of Comp. geometry app.

SRP: Rectangle

Separated responsibilities:



SRP: Rectangle

This design moves the computational portions of Rectangle into the GeometricRectangle class

Now changes made to the way rectangles are rendered cannot affect the Computational Geometry Application

SRP : Rectangle

In the context of the SRP, we define a responsibility to be a reason for change. If you can think of more than one motive for changing a class, that class has more than one responsibility.

SRP: Modem

Another example

```
public interface Modem{  
    public void Dial(string pno);  
    public void Hangup();  
    public void Send(char c);  
    public char Recv();  
}
```

SRP: Modem

However, two responsibilities are being shown here.

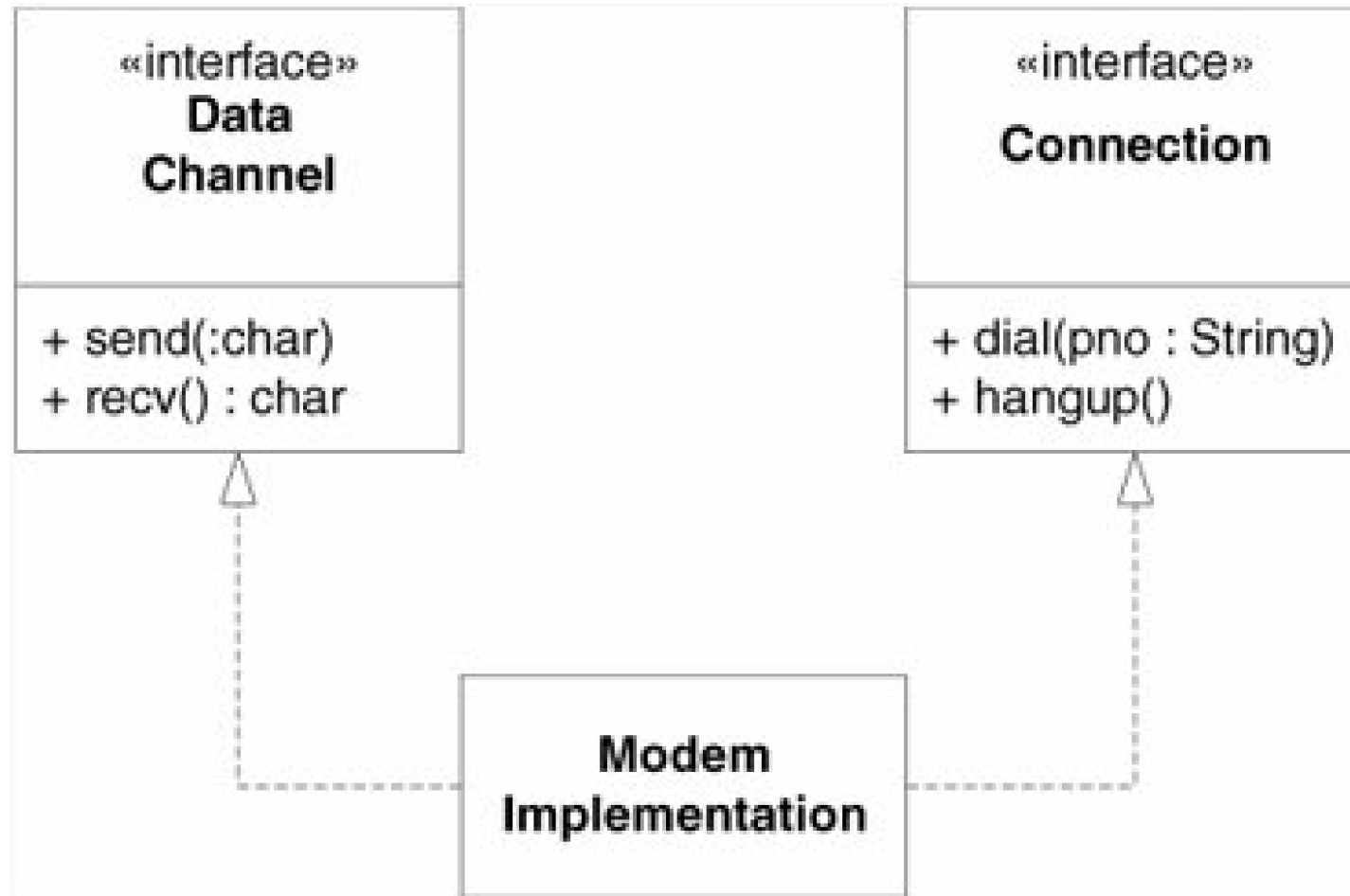
- connection management.

The `dial` and `hangup` functions manage the connection of the modem

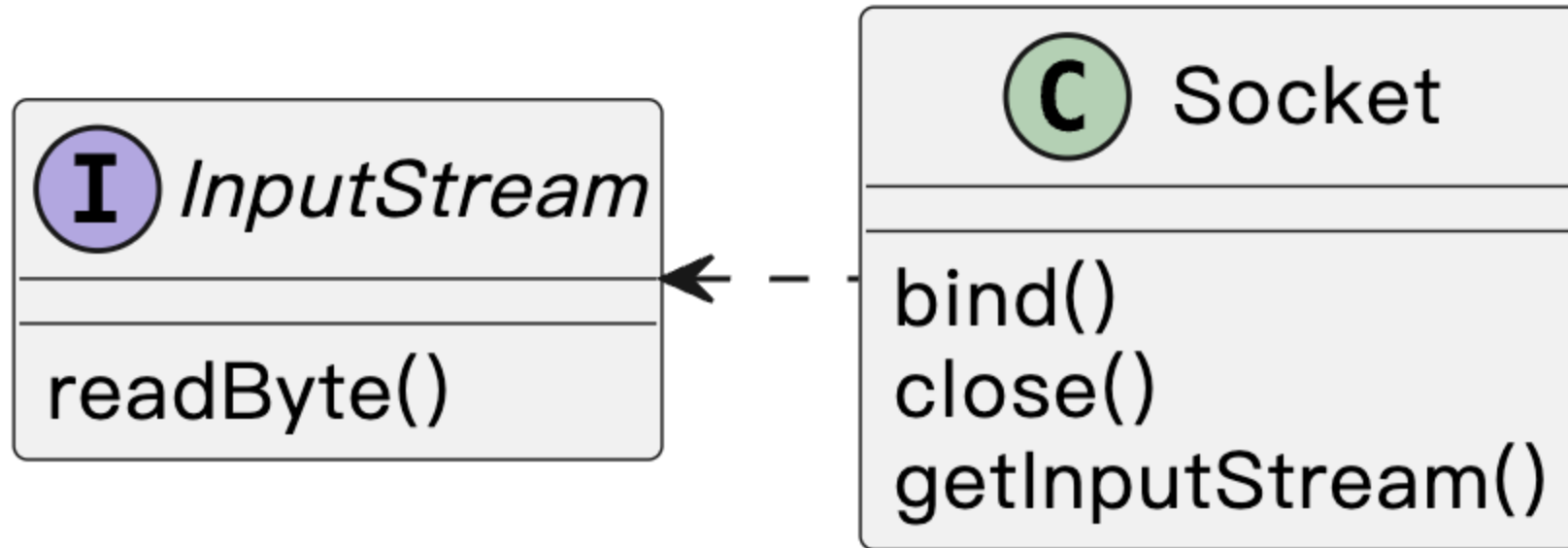
- data communication

the `send` and ``recv` functions communicate data.

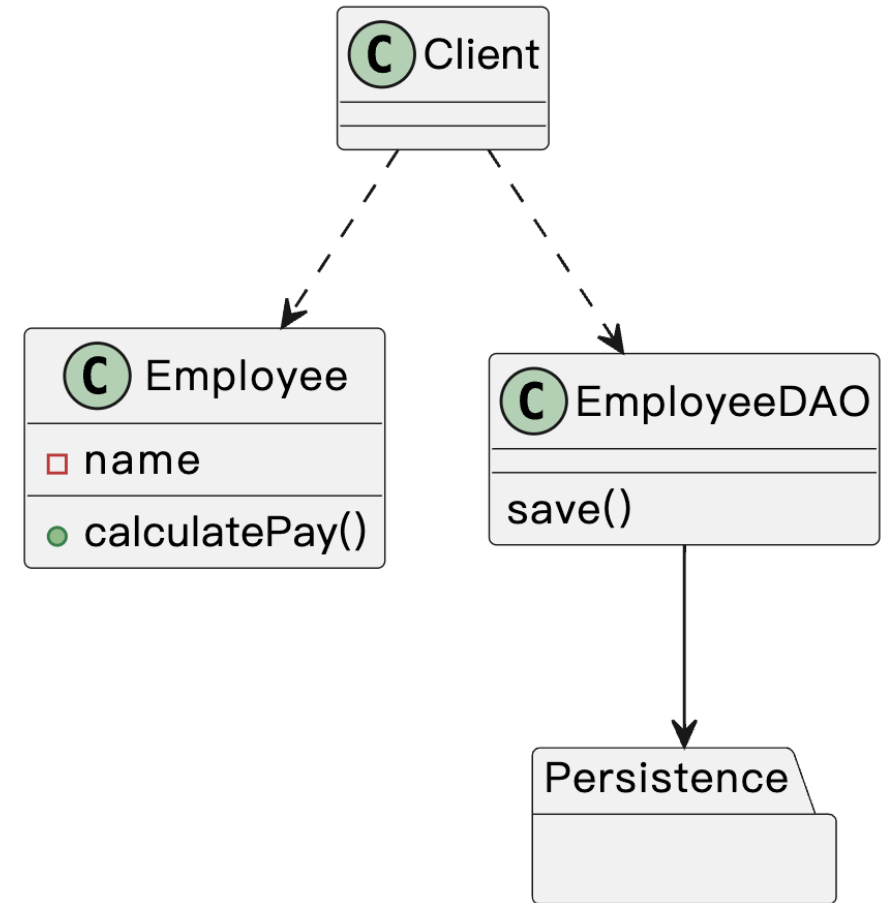
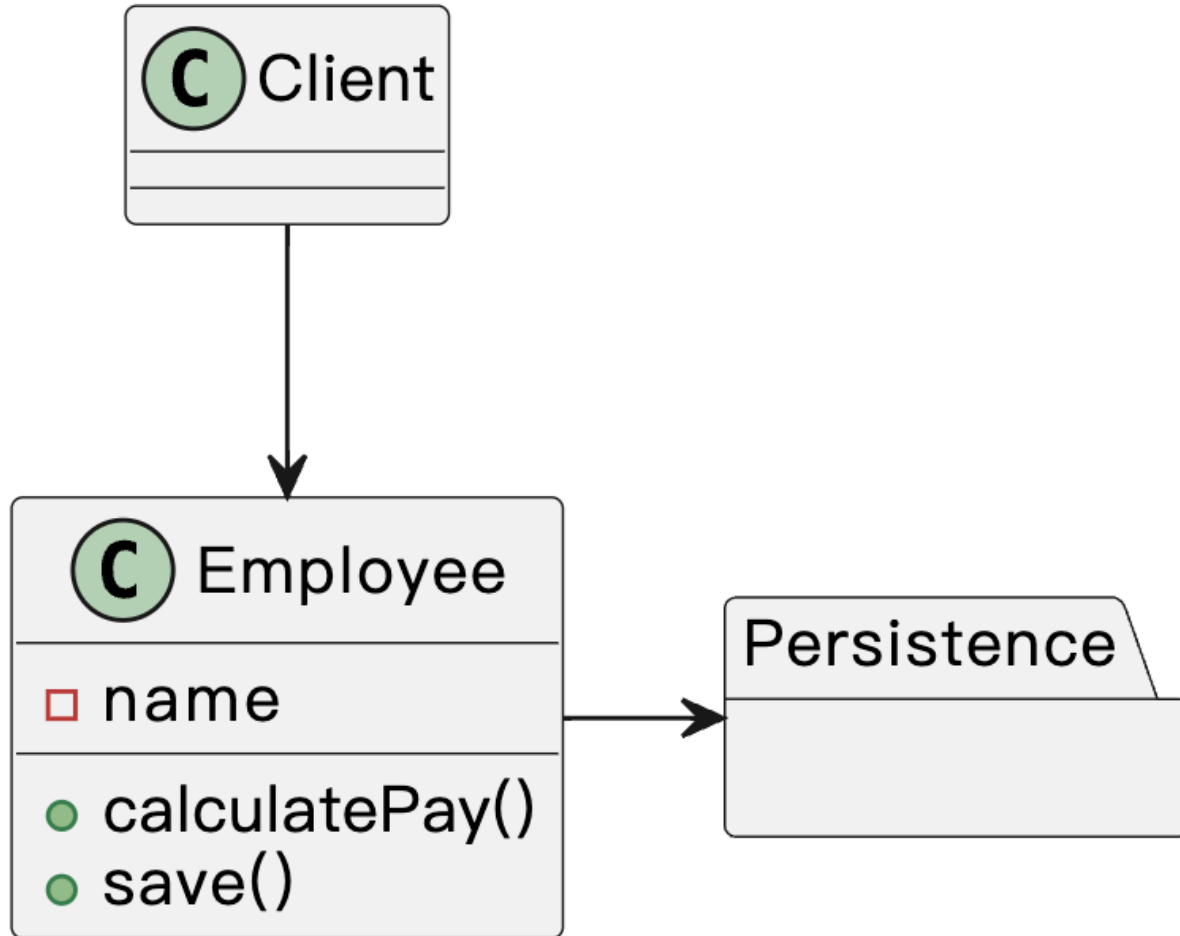
SRP: Modem



SRP : Java File/InputStream



SRP : Persistence



SRP : Summary

- The Single-Responsibility Principle is one of the simplest of the principles but one of the most difficult to get right.
- Conjoining responsibilities is something that we do naturally.
- Finding and separating those responsibilities is much of what software design is really about.

OCP: Open-Closed Principle

You should be able to extend a classes behavior, without modifying it.

OCP

- "All systems change during their life cycles." (IvarJacobson).
- "Software entities should be open for extension, butclosed for modification." (variation on BertrandMeyer's idea).
- Goal: avoid a "cascade of changes to dependent modules"
- Whe requirements change, you extend the behavior, not changing the old code

OCP

- Open for extension.

This means that the behavior of the module can be extended. As the requirements of the application change, we can extend the module with new behaviors that satisfy those changes.

- Closed for modification.

Extending the behavior of a module does not result in changes to the source, or binary, code of the module.

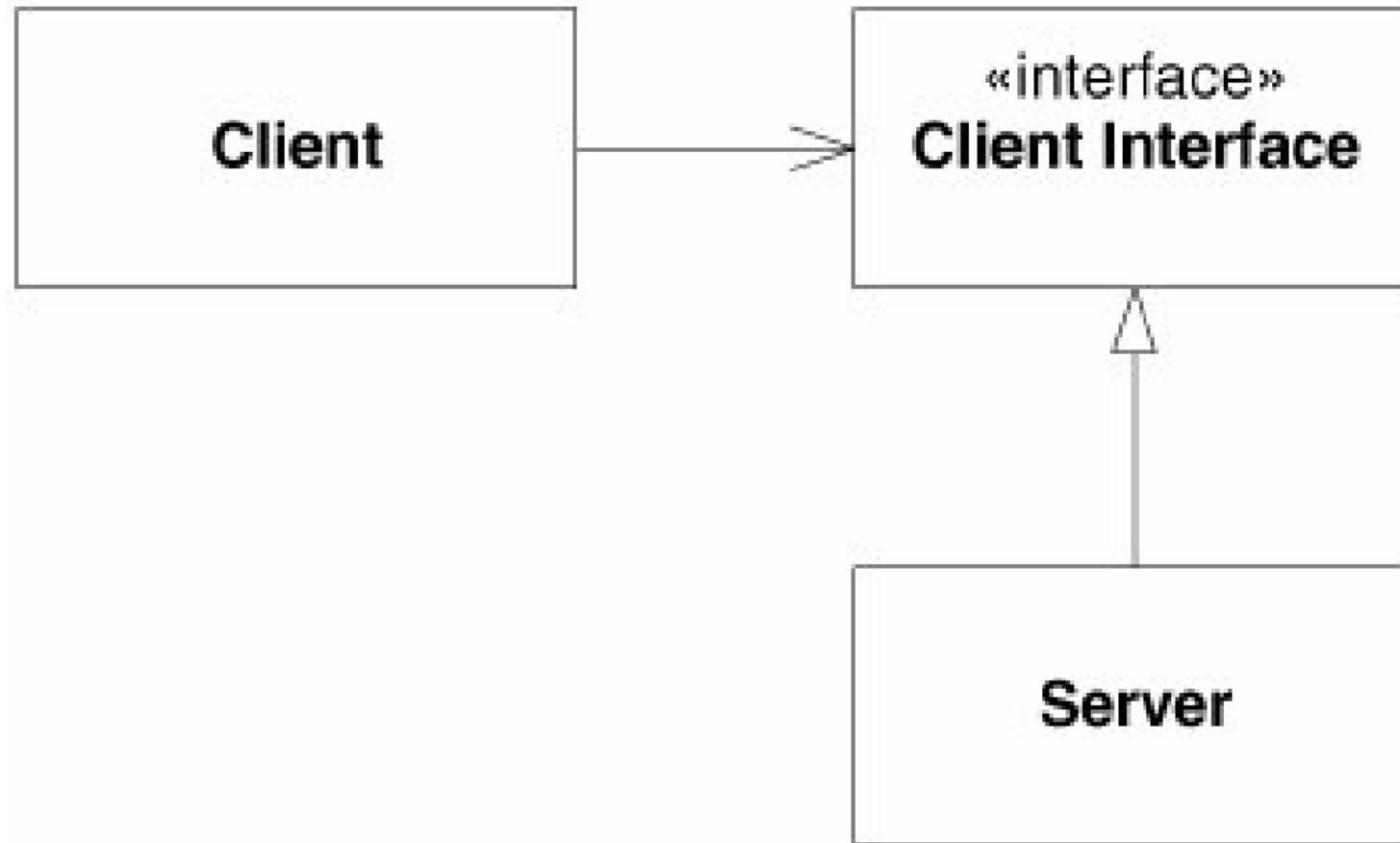
OCP : Client-Server



- Both the Client and Server classes are concrete.
- The Client class uses the Server class.
- If we want for a Client object to use a different serverobject, the Client class must be changed to name the new server class.

OCP : Client-Server

new design that conforms to the OCP

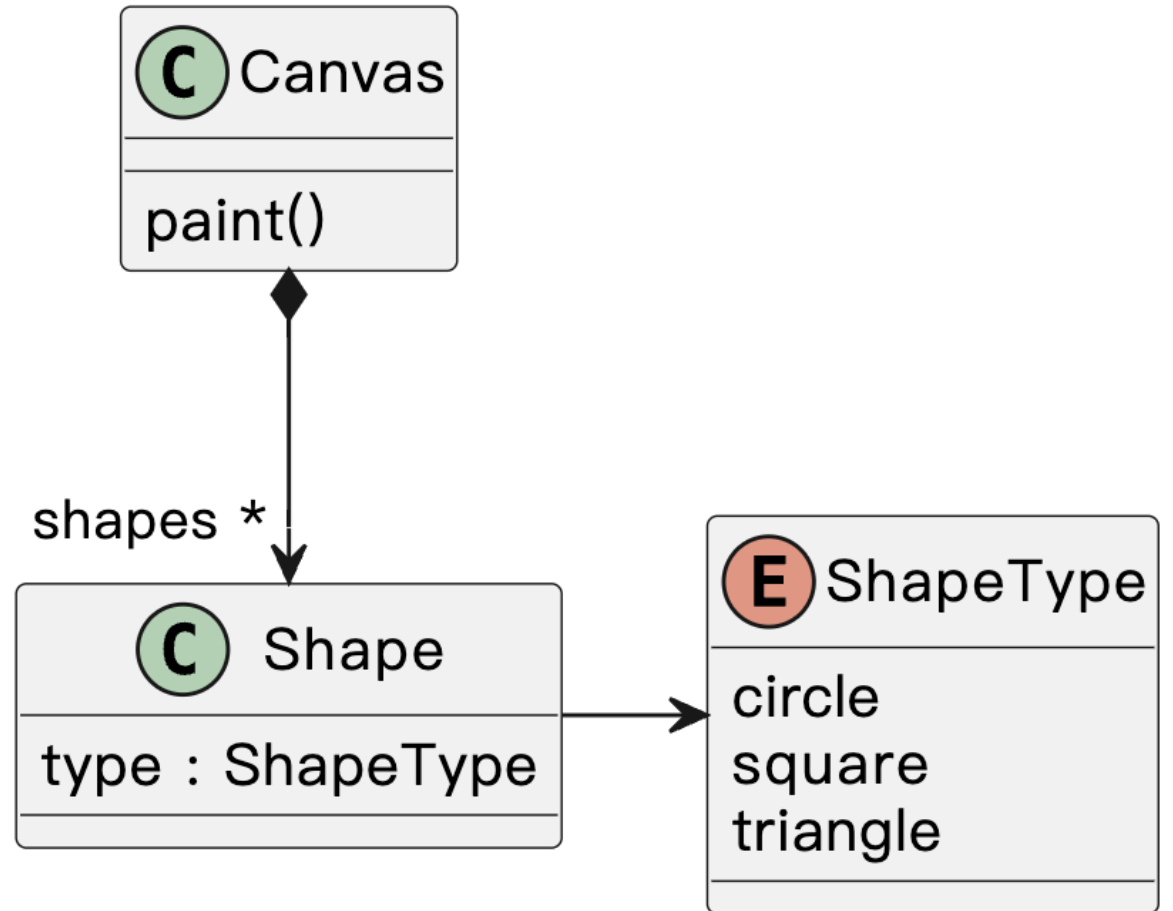


OCP : Client-Server

- In this case, the ClientInterface class is abstract with abstract member functions.
- The Client will be using objects of the derivative Server class.
- If we want Client objects to use a different server class, a new derivative of the ClientInterface class can be created. The Client class can remain unchanged.

OCP : Shape

```
public class Canvas{  
    public void paint(){  
        for(Shape shape : shapes){  
            switch(shape.type){  
                case circle: ...  
                case square: ...  
                case triangle: ...  
            }  
        }  
    }  
}
```



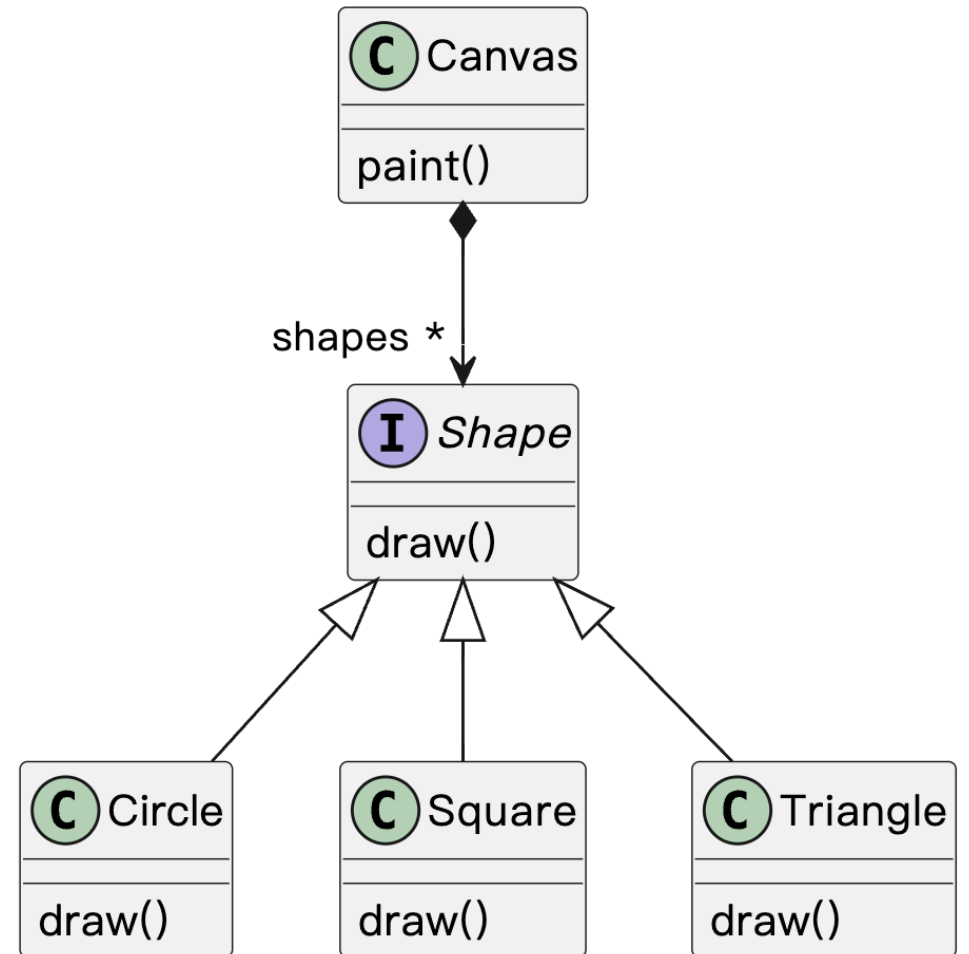
OCP : Shape

Because it cannot be closed against new kinds of shapes, the function paint does not conform to OCP.

In real life, the switch statement in the DrawAllShapes function would be repeated over and over again in various functions all through the application, each one doing something a little different.

OCP : Shape refactored

```
public class Canvas{  
    public void paint(){  
        for(Shape shape : shapes){  
            shape.draw()  
        }  
    }  
}
```



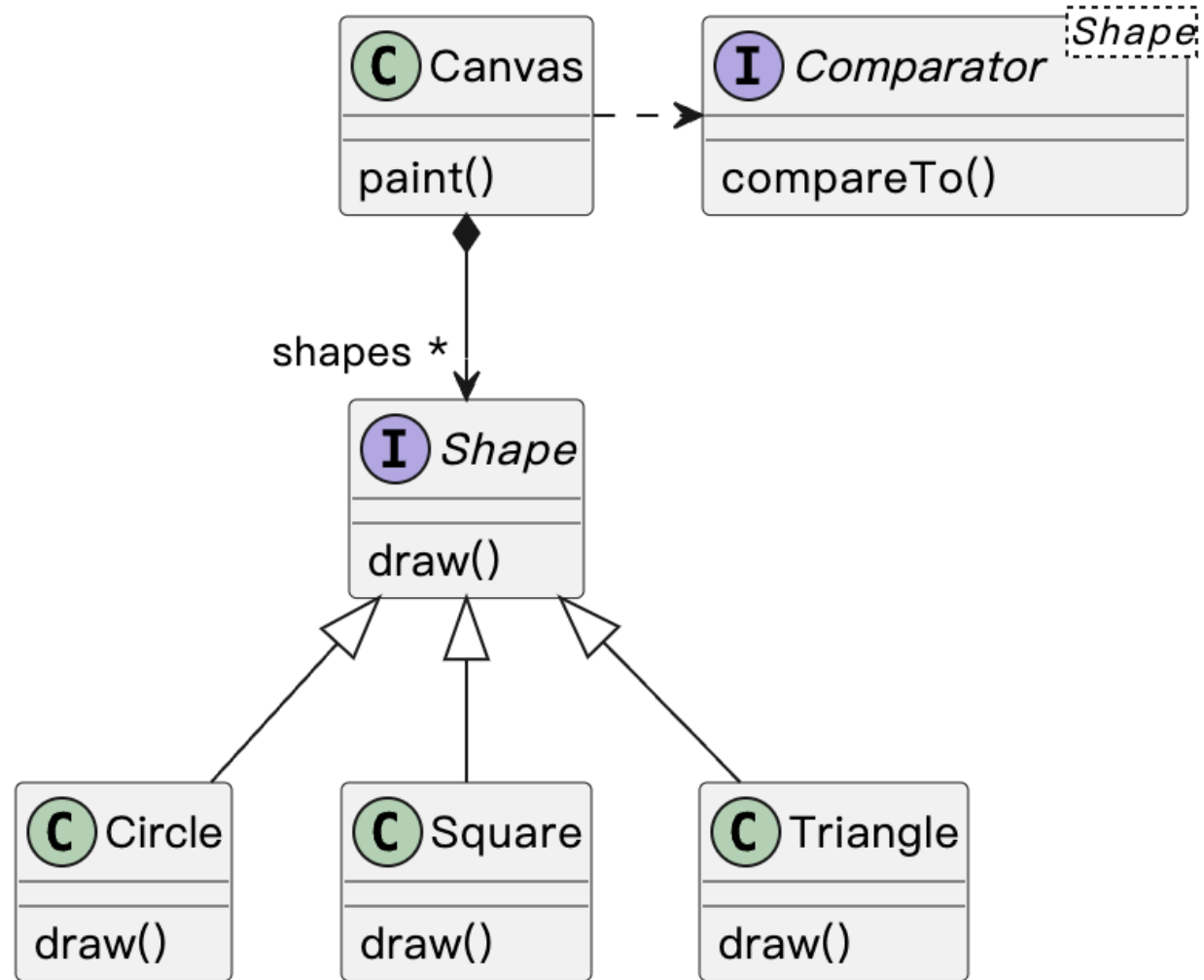
OCP: Shape

If user want to draw cirles before any other shapes.

OCP : on Anticipation of changes

- Putting the "Hooks" In
- Fool me once
- Stimulating change

OCP: Shape with comparator



OCP: Summary

- OCP is "at the heart of OOD"
- Simply using an OOP is not enough: Need dedication to apply abstraction
- OCP can greatly enhance reusability and maintainability